N053 Cloud Run으로 무중단 배포하기_해설서

Chapter01

챕터 01. Cloud Run으로 무중단 배포하기

1. 학습 목적

이번 강의의 핵심은 새로운 버전의 AI 서비스를 배포하더라도 사용자가 오류를 겪지 않도록 만드는 방법을 배우는 것입니다.

지금까지는 코드를 수정하고 테스트한 뒤 배포하는 방법을 배웠습니다. 이제는 한 단계 더 나아가 배포 과정에서 발생할 수 있는 문제를 예방하고, 새 버전에 문제가 있으면 이전 버전으로 되돌리는 방법까지 익힙니다.

2. 중학생도 이해할 수 있는 설명

식당에서 새로운 메뉴를 출시한다고 생각해 보겠습니다.

- 기존 버전: 현재 손님에게 제공하고 있는 메뉴
- 새 버전: 새롭게 개발한 메뉴
- 무중단 배포: 새 메뉴를 준비하는 동안에도 기존 메뉴를 계속 제공하는 것
- 롤백: 새 메뉴에 문제가 있으면 기존 메뉴로 되돌리는 것

새 메뉴를 모든 손님에게 한 번에 제공했다가 문제가 생기면 피해가 큽니다. 먼저 직원이 시험해 보고, 일부 손님에게 제공한 뒤 문제가 없으면 전체로 확대하는 것이 더 안전합니다.

AI 서비스도 같은 원리입니다.

3. 핵심 개념 5가지

① 리비전(Revision)

Cloud Run에서 배포된 서비스의 특정 버전입니다. 새 코드를 배포하면 일반적으로 새로운 리비전이 만들어집니다. 기존 리비전은 남아 있으므로 문제가 생겼을 때 이전 버전으로 되돌아갈 수 있습니다.

② 트래픽 설정(Traffic Splitting)

사용자 요청을 각 리비전에 몇 퍼센트씩 보낼지 결정합니다. 기존 버전에 90%, 새 버전에 10%를 보내는 식으로 운영할 수 있습니다.

③ 태그 URL(Tag URL)

새 리비전에 별도의 주소를 부여해 일반 사용자의 트래픽을 바꾸지 않고도 새 버전을 직접 시험할 수 있게 합니다. 시험 결과가 좋으면 실제 사용자 트래픽을 새 버전으로 전환합니다.

④ 스타트업 프로브(Startup Probe)

새 버전의 프로그램이 시작되는 데 시간이 걸릴 때, 실제로 준비되었는지 확인하는 장치입니다. 준비가 끝나기 전에 요청을 처리하려다 오류가 나는 상황을 방지하는 데 도움이 됩니다.

⑤ Supabase와 시크릿(Secret)

AI 친구 에이전트의 대화 기록을 데이터베이스에 저장합니다. Supabase 접속에 필요한 비밀 키는 코드에 직접 적지 않고 시크릿으로 안전하게 관리해 새 리비전에서도 사용할 수 있게 합니다.

4. 블루그린 배포와 카나리 배포

블루그린 배포 (Blue-Green)

What Is Blue-Green Deployment? - Semaphore

semaphore.io

기존 버전(Blue)과 새 버전(Green)을 준비해 두고 새 버전을 충분히 시험한 후 트래픽을 전환합니다.

예시: 새 버전으로 일반 트래픽을 보내기 전에 태그 URL로 먼저 확인하고, 문제가 없으면 새 버전으로 전환합니다.

- 장점: 전환 전 충분히 검증할 수 있고 롤백이 쉽습니다.
- 주의점: 새 버전이 데이터베이스 구조나 대화 처리 방식까지 바꾼다면 기존 버전과의 호환성을 확인해야 합니다.

카나리 배포 (Canary)

How to Use Python to Create and Implement Microservices

sayonetech.com

새 버전에 소량의 트래픽만 보내고 실제 사용 환경에서 문제가 없는지 확인한 뒤 점차 확대합니다.

예시: 기존 버전 90%, 새 버전 10%로 시작해 오류율과 응답 품질을 확인합니다.

- 장점: 문제가 생겨도 새 버전에 노출되는 요청을 제한할 수 있습니다.
- 주의점: 10%의 요청만 시험하면 드물게 발생하는 오류를 발견하지 못할 수도 있습니다.

5. 실제 서비스에 적용하는 순서

이번 실습의 배포 흐름은 다음과 같습니다.

1. 코드 변경 후 PR 머지

새 기능이나 수정 사항을 반영합니다.

2. CI 테스트 및 이미지 생성

테스트를 통과한 코드로 컨테이너 이미지를 만듭니다.

3. Cloud Run에 트래픽 0%로 배포

새 리비전을 준비하되 일반 사용자 트래픽은 보내지 않습니다.

4. 준비 상태 확인 및 스모크 테스트

서비스가 시작되고 핵심 기능이 정상 작동하는지 확인합니다.

5. 트래픽 전환

전체 전환 또는 일부 트래픽만 보내는 카나리 방식을 선택합니다.

6. 오류 감시 및 롤백

문제가 생기면 기존 리비전으로 되돌리고 결과를 기록합니다.

여기서 중요한 점은 이미지가 만들어졌다는 것과 서비스가 정상적으로 준비되었다는 것은 서로 다르다는 사실입니다. 따라서 새 리비전을 배포한 뒤 실제 요청을 보내 확인하는 과정이 필요합니다.

또한 트래픽 0%는 일반 트래픽을 보내지 않는다는 뜻이지, 새 버전이 자동으로 검증되었다는 뜻은 아닙니다. 태그 URL을 이용한 시험과 스모크 테스트를 별도로 수행해야 합니다.

6. AI 개발자와 1인 창업가가 알아야 하는 이유

이 기술은 단순한 서버 운영 지식이 아닙니다. 직접 운영할 AI 서비스의 신뢰도를 높이는 사업 역량입니다.

- My-Story-Maker(MSM): 사용자의 글과 이야기를 분석하는 기능이 잘못 배포되어도 서비스 중단과 데이터 손실 위험을 줄이는 데 활용할 수 있습니다.
- 뉴스레터 에이전트: 예약된 시간에 실행되는 서비스의 새 버전을 먼저 시험하고, 발행 오류가 발견되면 이전 버전으로 되돌릴 수 있습니다.

- AI 컨설팅 서비스: 고객별로 다른 에이전트를 운영할 때 배포 절차를 표준화하면 운영 위험을 줄이고 유지보수 비용을 관리하기 쉽습니다.

특히 고객의 대화 기록을 저장하는 AI 서비스에서는 코드뿐 아니라 데이터의 안전성, 버전 간 호환성, 오류 발생 시 복구 방법까지 함께 설계해야 합니다.

7. 시작 질문 모범 답안

한 번에 전환하는 경우에는 새 버전의 오류가 전체 사용자에게 영향을 줄 수 있습니다. 오류를 발견하기까지 시간이 걸리면 피해 범위가 커질 수 있습니다.

반면 트래픽의 10%만 새 버전으로 보내는 경우에는 새 버전을 사용하는 일부 사용자에게 문제가 집중되므로 초기 피해를 제한할 수 있습니다. 다만 10%의 사용자에게서 오류가 실제로 발생해야 발견할 수 있으므로, 오류 감시와 충분한 테스트가 필요합니다.

8. 반드시 기억할 것

- 리비전: 배포된 서비스의 버전
- 트래픽 설정: 각 버전에 요청을 보내는 비율
- 태그 URL: 일반 트래픽을 전환하기 전에 새 버전을 시험하는 별도 주소
- 스타트업 프로브: 애플리케이션이 시작될 준비가 되었는지 확인하는 장치
- 블루그린: 새 버전을 충분히 확인한 뒤 전환
- 카나리: 일부 트래픽부터 시작해 단계적으로 확대
- 자동 배포: 새 버전 배포 → 테스트 → 전환 → 실패 시 롤백을 자동화

마지막으로, 트래픽 비율을 나누더라도 같은 사용자의 연속 요청이 반드시 같은 리비전으로 가는 것은 아닙니다. 에이전트의 대화 상태를 데이터베이스에 저장하고 버전 간 호환성을 확보하는 것이 중요한 이유입니다.

9. 한 문장 요약

무중단 배포는 새 버전을 단순히 올리는 것이 아니라, 준비 상태를 확인하고 소량의 트래픽부터 검증한 뒤 안전하게 전환하며, 문제가 생기면 신속하게 복구하는 과정입니다.

Chapter02

챕터 02. 로드밸런서를 Cloud Run이 맡는다

1. 학습 목적

이번 강의의 핵심은 Cloud Run이 로드밸런서 역할을 대신하며, 실제 배포는 리비전을 만들고 사용자 요청이 어느 리비전으로 갈지 결정하는 과정이라는 점을 이해하는 것입니다.

지난 챕터에서는 무중단 배포의 전체 흐름을 배웠습니다. 이번에는 Cloud Run이 어떻게 요청을 새 버전으로 옮기는지, 롤백 후 왜 새 버전이 배포되어도 사용자에게 보이지 않을 수 있는지 살펴봅니다.

2. 중학생도 이해할 수 있는 설명

식당 입구에 손님을 안내하는 직원이 있다고 생각해 보겠습니다.

- 로드밸런서: 손님을 어느 식당으로 안내할지 결정하는 직원
- 리비전: 서로 다른 버전의 식당
- 트래픽 설정: 각 식당에 보내는 손님의 비율
- 배포: 새 식당을 준비하고 손님을 보내는 방식을 변경하는 것
- 롤백: 문제가 생기면 손님을 이전 식당으로 다시 안내하는 것

직접 서버를 운영하면 Nginx 같은 프로그램을 설치해 이 안내 직원 역할을 맡겨야 합니다. 하지만 Cloud Run에서는 Google Cloud가 이 역할을 관리하므로, 개발자는 리비전과 트래픽 설정에 집중할 수 있습니다.

3. 핵심 개념 정리

① 리비전(Revision)

Cloud Run에 배포된 특정 버전입니다. 새 버전을 배포하면 새로운 리비전이 생성되고, 이전 리비전도 남아 있을 수 있습니다.

② 트래픽 설정(Traffic)

사용자 요청을 리비전별로 얼마나 보낼지 정합니다. 예를 들어 기존 버전에 90%, 새 버전에 10%를 보내는 것이 카나리 배포입니다.

③ 스타트업 프로브(Startup Probe)

컨테이너가 단순히 실행되었는지만 보는 것이 아니라 애플리케이션이 요청을 처리할 준비가 되었는지 확인하도록 구성할 수 있습니다. 준비되지 않은 버전으로 너무 일찍 요청이 이동하는 문제를 줄입니다.

④ 롤백(Rollback)

문제가 생기면 이전 리비전으로 트래픽을 돌립니다. 새 이미지를 다시 만들 필요 없이 트래픽 설정을 변경하는 방식입니다. 다만 이전 버전의 정상 작동 여부는 확인해야 합니다.

⑤ 트래픽 고정(Traffic Pinning)

트래픽을 특정 리비전에 고정하면 다음 버전이 배포되어도 요청이 자동으로 새 버전으로 이동하지 않을 수 있습니다. 롤백 후 특히 주의해야 하는 부분입니다.

4. 기본 배포가 진행되는 순서

새 리비전 생성

스타트업 프로브 통과

새 인스턴스가 요청을 처리할 준비가 되었는지 확인

새 리비전으로 트래픽 전환

기본 배포는 새 버전으로 100% 전환

이전 인스턴스 종료

종료 신호를 보내고 진행 중인 요청을 마칠 기회를 제공

여기서 주의할 점은 포트가 열렸다는 사실과 서비스가 완전히 준비되었다는 사실은 다르다는 것입니다.

예를 들어 AI 에이전트가 실행된 직후에는 포트가 열려 있어도 모델 설정이나 색인을 불러오는 중일 수 있습니다. 이때 준비 상태를 제대로 확인하지 않으면 새 버전으로 요청이 이동한 직후 오류가 발생할 수 있습니다.

또한 정상 종료를 위한 대기 시간은 모든 요청의 완료를 무조건 보장하는 시간이 아닙니다. 애플리케이션이 종료 신호에 적절히 대응하고 제한 시간 안에 요청 처리를 마쳐야 합니다.

5. 블루그린·카나리·롤백 명령 비교

방식	핵심 명령	의미
기본 배포	`gcloud run services update ...`	기본 트래픽 전환 정책에 따라 새 버전을 배포합니다.
블루그린 준비	`--no-traffic --tag green`	일반 트래픽을 보내지 않고 새 리비전을 준비해 별도로 시험합니다.
블루그린 전환	`--to-tags green=100`	태그가 붙은 리비전으로 트래픽을 전환합니다.
카나리	`--to-tags canary=10`	새 버전에 트래픽 10%를 보냅니다.
롤백	`--to-revisions 이전리비전=100`	기존 리비전으로 트래픽을 되돌립니다.
최신 버전 추적	`--to-latest`	가장 최근 리비전으로 트래픽을 보내도록 설정합니다.

태그 URL은 일반 트래픽 설정과 별개로 특정 리비전을 시험할 수 있는 주소입니다. 단, 서비스가 공개되어 있다면 이 주소도 외부에 노출될 수 있으므로 민감한 서비스는 접근 통제를 고려해야 합니다.

6. 가장 중요한 함정: 트래픽 고정

이번 강의에서 반드시 기억해야 할 부분입니다.

트래픽 고정이 발생하는 과정

1단계. 정상 배포

리비전 2

리비전 2가 사용자 트래픽을 받고 있습니다.

2단계. 문제 발생 후 롤백

리비전 1

트래픽을 리비전 1에 100%로 고정합니다.

3단계. 버그 수정 후 재배포

리비전 3 생성

새 리비전은 생성되었지만 트래픽 설정이 자동으로 바뀌지 않아 사용자는 여전히 리비전 1을 이용할 수 있습니다.

4단계. 최신 버전으로 트래픽 전환

리비전 3

- `-to-latest`를 사용해 최신 리비전으로 트래픽을 보내도록 설정합니다.

이 상황은 배포 성공과 사용자에게 새 버전이 제공되는 것은 서로 다른 문제라는 점을 보여줍니다.

CI 테스트가 통과하고 이미지가 만들어졌더라도, 실제 서비스 주소에서 `/version`을 확인해 새 리비전이 응답하는지 점검해야 합니다.

7. AI 개발자와 1인 창업가에게 주는 의미

이 개념은 실제 AI 서비스 운영에서 직접 활용할 수 있습니다.

- MSM(My-Story-Maker): 글 분석이나 웹소설 생성 기능을 변경할 때 새 버전을 먼저 검증하고, 오류가 있으면 기존 버전으로 빠르게 되돌릴 수 있습니다.

- 뉴스레터 에이전트: 정해진 시간에 뉴스레터를 수집·분석·발행하는 서비스에서 배포 후 실제 실행 버전을 확인할 수 있습니다.
- AI 컨설팅 서비스: 여러 고객의 에이전트를 운영할 때 배포 절차를 표준화하고, 배포 성공 여부와 사용자 서비스 상태를 분리해 관리할 수 있습니다.

앞으로는 단순히 "배포가 성공했습니다"라는 메시지만 보는 것이 아니라 다음 세 가지를 함께 확인하는 습관이 중요합니다.

1. 새 리비전이 정상적으로 생성되었는가?
2. 새 리비전이 실제 요청을 받고 있는가?
3. 요청 처리 결과가 정상인가?

8. 객관식 퀴즈 정답

정답: 2번

롤백할 때 트래픽을 이전 리비전에 고정했기 때문에 새 리비전은 생성되었어도 사용자 트래픽을 받지 못한 것입니다.

이미지를 만들지 못했다면 배포 워크플로의 이미지 생성 단계에서 문제가 나타날 수 있습니다. 스타트업 프로브 실패가 반드시 리비전 삭제를 의미하지는 않으며, 태그 URL이 일반 서비스 주소를 자동으로 대체하는 것도 아닙니다.

9. 반드시 기억할 것

- Cloud Run은 서비스 요청을 리비전으로 전달하는 역할을 관리합니다.
- 무중단 배포의 핵심은 새 리비전을 준비하고 트래픽을 안전하게 전환하는 것입니다.
- 스타트업 프로브는 포트 개방과 애플리케이션 준비 상태의 차이를 고려해야 합니다.
- 롤백은 트래픽을 이전 리비전으로 되돌리는 것이며, 이전 버전의 정상 작동을 자동으로 보장하지 않습니다.
- 트래픽을 특정 리비전에 고정하면 이후 배포에서 새 버전으로 자동 전환되지 않을 수 있습니다.
- 배포 성공 메시지만 믿지 말고 실제 서비스의 `/version`과 요청 결과를 확인해야 합니다.

10. 한 문장 요약

Cloud Run 무중단 배포의 핵심은 리비전을 안전하게 준비하고 트래픽을 제어하는 것이며, 롤백 이후에는 트래픽 고정 때문에 새 버전이 실제 사용자에게 제공되지 않는 상황까지 점검해야 한다는 것입니다.

Chapter03

챕터 03. 시작 상태 맞추기

1. 학습 목적

이번 강의의 핵심은 Cloud Run의 리비전이 바뀌어도 사용자의 대화와 기억이 사라지지 않도록 데이터를 데이터베이스에 저장하는 것입니다.

무중단 배포에서는 이전 버전과 새 버전이 동시에 사용자 요청을 처리할 수 있습니다. 그런데 대화 기록을 각 인스턴스의 메모리에만 저장하면 버전마다 서로 다른 대화를 보게 될 수 있습니다.

이를 해결하기 위해 Supabase를 연결하고, 데이터 저장 위치와 로그인 방식, 비밀 키 관리 방법을 설정합니다.

2. 중학생도 이해할 수 있는 설명

여러 직원이 근무하는 상담센터를 생각해 보겠습니다.

- 인스턴스 메모리: 직원 한 명이 자기 책상에만 보관한 상담 기록
- Supabase 데이터베이스: 모든 직원이 함께 보는 중앙 상담 기록실
- 사용자 인증(Auth): 상담을 받으러 온 사람이 누구인지 확인하는 절차
- 무상태(Stateless): 직원이 바뀌어도 중앙 기록실에서 상담 기록을 찾아 이어서 처리하는 구조
- Secret Manager: 중요한 열쇠를 안전하게 보관하는 금고

기존에는 상담 직원이 바뀌면 책상에 있던 기록을 잃어버리는 문제가 있었습니다. 이제 상담 기록을 중앙 데이터베이스에 저장하면 새로운 직원이 와도 이전 기록을 이어서 확인할 수 있습니다.

3. 핵심 개념 정리

① Supabase

PostgreSQL 데이터베이스와 사용자 인증 기능을 제공하는 관리형 서비스입니다. 이번 실습에서는 AI 친구 에이전트의 대화 기록을 저장합니다.

② 익명 로그인(Anonymous Sign-in)

사용자가 이메일이나 비밀번호를 입력하지 않고도 게스트 계정을 만들어 서비스를 사용할 수 있는 기능입니다. 브라우저마다 별도의 게스트 사용자로 시작할 수 있습니다.

③ RLS(Row Level Security)

데이터베이스의 행마다 누가 읽거나 쓸 수 있는지 제한하는 보안 기능입니다. 예를 들어 사용자 A가 사용자 B의 대화 기록을 조회하지 못하도록 통제할 수 있습니다. 실제 보호를 위해서는 정책 설정도 올바르게 되어 있어야 합니다.

④ Publishable Key와 Secret Key

Publishable Key는 브라우저에 전달할 수 있는 공개용 키입니다. Secret Key는 강력한 권한을 가질 수 있으므로 서버에서만 사용하고 Secret Manager에 보관해야 합니다.

⑤ 무상태 서비스(Stateless Service)

인스턴스가 바뀌더라도 필요한 대화 기록을 데이터베이스에서 다시 읽을 수 있는 구조입니다. 여러 리비전이 함께 요청을 처리해도 같은 저장소를 이용할 수 있습니다.

4. 전체 구성 이해하기

사용자 브라우저

게스트 로그인 · 대화 입력

Cloud Run

기존 리비전 또는 새 리비전에서 요청 처리

Supabase

사용자 · 캐릭터 · 대화 기록 저장

두 리비전이 같은 데이터베이스를 사용하므로 인스턴스가 바뀌어도 대화를 이어갈 수 있습니다.

이번 실습에서는 환경을 두 개로 분리합니다.

구분	Cloud Run 서비스	Supabase
시험 환경	`friend-agent-staging`	시험용 프로젝트
운영 환경	`friend-agent`	`friend-agent-production`

이렇게 분리하면 시험 중 만들어진 대화 기록과 운영 데이터가 섞이는 것을 방지할 수 있습니다.

5. 실습 진행 순서

1. 시작 상태 확인: Google Cloud 프로젝트, Cloud Run 서비스, GitHub Actions 배포 설정 등이 준비되었는지 확인합니다.
2. Supabase 프로젝트 준비: 시험용 프로젝트와 운영용 프로젝트를 준비하고 필요한 테이블 및 익명 로그인을 설정합니다.
3. 비밀 키 보관: 각 환경의 Secret Key를 Google Cloud Secret Manager에 별도로 저장합니다.
4. staging 연결: 시험용 URL, Publishable Key, Secret Key를 staging 서비스에 연결합니다.
5. 데이터 저장 확인: 대화를 입력하고 Supabase의 `messages` 테이블에 기록이 생기는지 확인합니다.
6. 리비전 변경 확인: 새 리비전을 배포한 후에도 기존 대화가 남아 있는지 확인합니다.
7. production 연결: 운영 서비스에 운영용 Supabase 프로젝트를 연결합니다.

6. 반드시 주의할 보안 사항

이번 교안에서는 staging과 production이 같은 런타임 서비스 계정을 사용합니다. 따라서 두 시크릿에 모두 접근할 수 있다는 한계가 있습니다.

실제 서비스를 공개하기 전에는 환경별 서비스 계정을 분리하는 것이 바람직합니다.

- staging 서비스 계정 → staging Secret Key만 읽기
- production 서비스 계정 → production Secret Key만 읽기

또한 Secret Key는 명령어, 코드, 로그 또는 AI 코딩 에이전트의 대화에 직접 붙여 넣지 않는 것이 중요합니다. Secret Manager에는 키의 실제 값이 저장되고, Cloud Run에는 그 시크릿을 참조하도록 설정합니다.

7. AI 개발자와 1인 창업가에게 주는 의미

이 강의는 AI 서비스가 단순한 시연용 프로그램에서 실제 사용자의 데이터를 책임지는 서비스로 발전하는 데 필요한 기반을 다룹니다.

- MSM: 사용자가 작성한 글과 생성된 이야기를 리비전 변경 후에도 유지할 수 있습니다.
- AI 친구 에이전트: 캐릭터와 대화 기록을 지속적으로 저장해 사용자 경험을 유지할 수 있습니다.
- AI 컨설팅 서비스: 고객별 데이터를 분리하고 접근 권한을 관리하는 운영 구조를 설계할 수 있습니다.

특히 무중단 배포를 위해서는 서버를 두 개 실행하는 것만으로는 충분하지 않습니다. 데이터를 공유하는 구조와 사용자별 접근 권한을 함께 설계해야 합니다.

8. 객관식 퀴즈 정답

정답: 2번

Secret Key는 RLS 정책을 우회하거나 광범위한 데이터 접근 권한을 가질 수 있으므로 서버 측에서 안전하게 관리해야 합니다. Publishable Key는 브라우저에 전달할 수 있는 공개용 키입니다.

다만 Publishable Key가 공개 가능하다는 것은 데이터 접근까지 무제한으로 허용한다는 뜻이 아닙니다. 실제 데이터 보호는 RLS와 올바른 접근 정책에 달려 있습니다.

9. 반드시 기억할 것

- 인스턴스 메모리에만 저장한 대화는 새 리비전에서 이어지지 않을 수 있습니다.
- Supabase에 대화 기록을 저장하면 여러 리비전이 같은 데이터를 사용할 수 있습니다.
- 익명 로그인은 사용자가 간편하게 게스트로 시작할 수 있도록 합니다.
- RLS는 사용자별 데이터 접근을 제한하는 중요한 보안 장치입니다.
- Publishable Key와 Secret Key는 용도와 보관 방법이 다릅니다.
- staging과 production의 데이터 및 접근 권한을 분리해야 합니다.
- 데이터베이스를 공유할 때는 두 리비전이 호환되는 테이블 구조를 사용하는지도 확인해야 합니다.

10. 한 문장 요약

무중단 배포를 제대로 구현하려면 대화 기록을 인스턴스 메모리에서 공용 데이터베이스로 옮기고, 사용자 인증과 환경별 비밀 키 관리를 통해 데이터의 지속성과 보안을 함께 확보해야 합니다.

Chapter04

챕터 04. 요청 확인 스크립트

1. 학습 목적

이번 강의의 핵심은 배포가 성공했는지가 아니라, 배포하는 동안 실제 사용자 요청이 실패하지 않았는지를 측정하는 것입니다.

Cloud Run은 새 리비전이 준비되었다고 판단하면 트래픽을 전환합니다. 그런데 애플리케이션이 실제로 요청을 처리할 준비가 되지 않았다면 사용자는 오류를 경험할 수 있습니다.

이번 실습에서는 요청 확인 스크립트로 배포 전후의 응답을 측정하고, 스타트업 프로브를 개선해 이 문제를 해결합니다.

2. 중학생도 이해할 수 있는 설명

식당에서 새로운 주방을 열었다고 생각해 보겠습니다.

- 주방의 전등이 켜졌다고 요리를 바로 할 수 있는 것은 아닙니다.
- 재료와 조리 도구가 준비되어야 손님에게 음식을 제공할 수 있습니다.
- 준비가 끝나기 전에 손님을 새 주방으로 안내하면 주문이 실패할 수 있습니다.

Cloud Run의 기본 스타트업 프로브가 포트가 열렸는지만 확인한다면, 전등이 켜진 것만 보고 주방이 준비되었다고 판단하는 셈입니다.

`/health` 프로브를 활용하면 실제 서비스가 준비되었는지 확인한 뒤 트래픽을 전환하도록 구성할 수 있습니다.

3. 핵심 개념 정리

① 요청 확인 스크립트 — `watch_requests.py`

`/version` 주소에 0.2초마다 요청을 보내고 응답 코드, 리비전, 실패 시각을 기록합니다. 전체 요청 수와 실패 수를 확인해 배포 과정의 영향을 측정합니다.

② 콜드 스타트(Cold Start)

요청이 없어 쉬고 있던 서비스가 새로운 인스턴스를 시작하면서 응답이 늦어지는 현상입니다. 실습 스크립트는 2초 안에 응답이 없으면 실패로 기록하므로 배포와 무관한 콜드 스타트도 구분해야 합니다.

③ 스타트업 프로브(Startup Probe)

새 인스턴스가 시작되는 동안 준비 상태를 검사합니다. 포트 개방만 확인하는 대신 `/health`에 HTTP 요청을 보내도록 설정하면 애플리케이션이 준비되었는지 확인할 수 있습니다.

④ 무중단 배포 검증

새 리비전이 준비될 때까지 기존 리비전이 요청을 처리하도록 합니다. 사용자가 오류를 경험하지 않게 하는 대신 새 버전의 준비 시간이 길어질 수 있습니다.

4. 실습의 핵심: 세 가지 상황 비교

실험	설정	관찰할 결과
1. 기본 배포	`BANNER=측정1`	새 리비전으로 전환할 때 요청 실패가 발생하는지 확인
2. 준비가 늦은 버전	`SLOW_START_SECONDS=30`, `BANNER=측정2`	포트는 열렸지만 앱이 준비되지 않아 503 오류가 발생하는지 확인
3. `/health` 프로브	`/health` 프로브, `BANNER=측정3`	앱이 준비될 때까지 기존 리비전이 요청을 처리해 실패가 줄어드는지 확인

실험 2에서는 포트가 열렸다는 이유로 트래픽이 너무 일찍 전환될 수 있습니다. 실험 3에서는 `/health`가 준비되지 않은 동안 실패 응답을 반환하므로 Cloud Run이 새 리비전을 준비 완료로 판단하지 않습니다.

다만 실제 측정 결과는 실행 환경과 요청 타이밍에 따라 달라질 수 있으므로, 교안의 예상 결과와 직접 관찰한 결과를 비교하는 것이 중요합니다.

5. 스타트업 프로브 설정 이해하기

이번 실습에서 사용하는 명령은 다음과 같습니다.

```
gcloud run services update friend-agent-staging \
  --region us-central1 \
  --startup-probe=httpGet.path=/health,periodSeconds=5,failureThreshold=12,timeoutSeco
nds=2 \
  --update-env-vars BANNER=측정3
```

각 설정의 의미는 다음과 같습니다.

- `httpGet.path=/health`: `/health` 주소로 준비 상태를 확인합니다.
- `periodSeconds=5`: 5초 간격으로 검사합니다.
- `failureThreshold=12`: 연속 실패 허용 횟수를 설정합니다.
- `timeoutSeconds=2`: 검사 한 번의 응답을 최대 2초 기다립니다.
- `BANNER=측정3`: 새 리비전을 구분하기 위한 문구입니다.

5초 간격으로 12번 실패한다고 해서 모든 환경에서 정확히 60초 후에 종료되는 것은 아닙니다. 검사 시작 시점과 각 검사에 걸린 시간 등도 영향을 줍니다.

또한 `/health`는 단순히 HTTP 200을 반환하는 주소가 아니라 서비스가 실제로 요청을 처리할 준비가 되었을 때 성공 응답을 반환하도록 구현되어야 합니다.

6. 자동 배포에도 설정을 유지하기

터미널에서 직접 설정한 프로브는 해당 서비스 설정에 반영됩니다. 그러나 이후 배포 워크플로에서도 같은 설정을 명시적으로 유지해야 환경별 배포 동작을 일관되게 관리할 수 있습니다.

`.github/workflows/deploy.yml`의 배포 단계 `flags`에 다음 옵션을 추가합니다.

```
flags: --service-account=${{ vars.RUNTIME_SA }} --startup-probe=httpGet.path=/health,p
eriodSeconds=5,failureThreshold=12,timeoutSeconds=2
```

기존 `--service-account` 설정은 유지해야 합니다. 이 수정은 작업 브랜치에서 진행하고 PR로 검토한 뒤 staging과 main에 반영합니다.

실험이 끝나면 staging 서비스의 실습용 환경변수도 정리합니다.

```
gcloud run services update friend-agent-staging \
  --region us-central1 \
  --remove-env-vars SLOW_START_SECONDS,BANNER
```

7. AI 개발자와 1인 창업가에게 주는 의미

이 강의에서 배우는 것은 단순한 명령어가 아니라 실제 서비스의 품질을 측정하고 개선하는 방법입니다.

- MSM: 글 분석이나 웹소설 생성 기능이 시작되는 동안 사용자가 오류를 경험하지 않도록 준비 상태를 관리할 수 있습니다.
- 뉴스레터 에이전트: 예약 실행 시간에 맞춰 서비스가 정상적으로 준비되었는지 확인할 수 있습니다.
- AI 컨설팅 서비스: 고객에게 서비스를 배포할 때 성공 여부뿐 아니라 오류율과 사용자 영향을 기록으로 제시할 수 있습니다.

특히 사업 운영에서는 "배포가 성공했습니다"보다 "배포 중 요청 600건 가운데 실패가 0건이었습니다"처럼 측정 가능한 결과가 더 유용합니다. 단, 실패 0건은 측정한 시간과 요청 범위에서만 의미가 있으며 모든 상황에서 오류가 없다는 보증은 아닙니다.

8. 객관식 퀴즈 정답

정답: 3번. 트래픽이 아직 옮겨 가지 않아 이전 리비전이 처리했습니다.

`/health` 프로브가 새 리비전의 준비 완료를 확인할 때까지 Cloud Run은 새 리비전으로 트래픽을 전환하지 않습니다. 따라서 준비가 늦어지는 동안 기존 리비전이 사용자 요청을 처리

합니다.

2번처럼 Cloud Run이 요청을 모아 두었다가 나중에 보내는 방식은 아닙니다. 핵심은 요청을 대기열에 쌓는 것이 아니라 기존 버전을 계속 사용하다가 새 버전이 준비되면 전환하는 것입니다.

9. 반드시 기억할 것

- 배포 성공과 사용자 요청 성공은 서로 다른 지표입니다.
- `watch_requests.py`로 응답 코드와 리비전별 응답 수를 확인할 수 있습니다.
- 콜드 스타트로 인한 지연과 실제 배포 오류를 구분해야 합니다.
- 포트 개방만으로 애플리케이션의 준비 완료를 판단하면 오류가 발생할 수 있습니다.
- `/health` 스타트업 프로브로 준비 상태를 확인한 뒤 트래픽을 전환하도록 구성할 수 있습니다.
- 프로브 설정을 배포 워크플로에도 반영해 이후 배포에서 일관성을 유지해야 합니다.
- 오류 0건이라는 결과는 측정 범위와 시간 조건을 함께 기록해야 정확하게 해석할 수 있습니다.

10. 한 문장 요약

무중단 배포는 새 버전을 만드는 것만으로 완성되지 않으며, 실제 요청을 측정하고 애플리케이션이 준비된 뒤 트래픽을 전환하도록 해야 사용자 오류를 줄일 수 있습니다.

Chapter05

/강의정리 챕터 05. 트래픽 0%로 새 리비전 올리기

1. 학습 목적

이번 강의의 핵심은 새 버전을 배포하더라도 곧바로 사용자에게 공개하지 않고, 먼저 확인한 다음 안전하게 전환하고 문제가 생기면 빠르게 되돌리는 방법을 배우는 것입니다.

Cloud Run에서 실습한 주요 개념은 다음과 같습니다.

- 블루그린 배포: 기존 버전과 새 버전을 준비하고, 확인 후 한 번에 전환합니다.
- 카나리 배포: 새 버전에 트래픽을 10%처럼 조금씩 보내며 문제가 없는지 확인합니다.

- 롤백: 문제가 생기면 이전 버전으로 트래픽을 되돌립니다.
- 트래픽 고정: 트래픽이 특정 리비전에 고정되어 있으면 새 리비전을 만들어도 자동으로 사용자에게 공개되지 않을 수 있습니다.
- 관측과 판단 기준: 오류율, 응답 속도, 핵심 기능, 인스턴스 상태, 사용자 반응과 비용을 기준으로 배포를 판단합니다.

2. 중학생도 이해하는 핵심 개념

2-1. 리비전이란?

리비전(Revision)은 Cloud Run에 배포된 서비스의 특정 버전입니다.

예를 들어 AI 에이전트 서비스를 운영한다고 가정하겠습니다.

리비전 A — 기존 버전

현재 정상 운영 중 · 트래픽 100%

리비전 B — 새 버전

테스트 중 · 트래픽 0% · green 태그

새 버전 B를 만들었다고 해서 기존 버전 A가 사라지는 것은 아닙니다. 두 버전을 모두 유지한 상태에서 새 버전을 시험할 수 있습니다.

2-2. 블루그린 배포란?

블루그린은 기존 버전과 새 버전을 나란히 준비하고, 새 버전의 정상 작동을 확인한 후 사용자의 요청이 향하는 대상을 한 번에 바꾸는 방식입니다.

① 기존 버전 운영

100%

기존 버전이 모든 요청을 처리합니다.

② 새 버전 테스트

0%

green 태그 URL로 새 버전만 따로 확인합니다.

③ 새 버전으로 전환

100%

문제가 없다면 새 버전으로 모든 요청을 보냅니다.

핵심은 새 버전을 만들 때와 사용자의 요청을 새 버전으로 보내는 시점을 분리하는 것입니다.

2-3. 카나리 배포란?

카나리는 새 버전으로 트래픽을 조금씩 옮기는 방식입니다.

- 10%: 일부 요청으로 먼저 확인
- 50%: 문제가 없다면 절반으로 확대
- 100%: 최종적으로 모든 요청을 새 버전으로 전환

단, 10%로 설정했다고 실제 요청 중 정확히 10%가 새 버전으로 가는 것은 아닙니다. 요청마다 분배되므로 실제 관측 비율은 8%, 13% 등으로 달라질 수 있습니다.

또한 요청 수가 적으면 카나리의 효과가 떨어집니다. 하루 30건의 요청을 받는 서비스에서 새 버전이 10%를 받으면 하루 평균 약 3건만 새 버전으로 들어갑니다. 이때 오류가 없었다고 해서 충분히 안전하다고 판단하기는 어렵습니다.

3. 비유로 이해하기

새로운 식당 메뉴를 출시한다고 생각해 보겠습니다.

- 기본 배포: 새 메뉴를 모든 손님에게 한꺼번에 제공합니다.
- 블루그린: 기존 메뉴는 계속 판매하면서 새 메뉴를 직원들이 먼저 시험한 뒤, 준비가 끝나면 한꺼번에 전환합니다.
- 카나리: 처음에는 손님 일부에게만 새 메뉴를 제공하고, 반응이 좋으면 점점 더 많은 손님에게 제공합니다.
- 롤백: 새 메뉴에 문제가 발견되면 기존 메뉴로 즉시 돌아갑니다.

여기서 중요한 것은 새 메뉴를 준비하는 일과 실제 손님에게 판매하는 일을 구분한다는 점입니다.

4. 실습 명령어와 활용법

4-1. 현재 운영 중인 리비전 확인

```
gcloud run services describe friend-agent-staging \
  --region us-central1 \
  --format "yaml(status.traffic)"
```

이 명령으로 현재 어떤 리비전이 얼마만큼의 트래픽을 받는지 확인합니다.

왜 먼저 확인할까요? 문제가 생겼을 때 돌아갈 이전 리비전의 이름을 알아야 하기 때문입니다.

4-2. 새 리비전을 트래픽 0%로 배포

```
gcloud run services update friend-agent-staging \
  --region us-central1 \
  --update-env-vars "BANNER=GREEN" \
  --no-traffic \
  --tag green
```

각 옵션의 의미는 다음과 같습니다.

옵션	의미
`--update-env-vars`	환경변수 `BANNER` 값을 변경합니다.
`--no-traffic`	새 리비전으로 일반 트래픽을 보내지 않습니다.
`--tag green`	새 리비전을 확인할 수 있는 태그 URL을 만듭니다.

가장 중요한 옵션은 `--no-traffic`입니다. 이것이 빠지면 새 버전이 곧바로 일반 트래픽을 받을 수 있습니다.

배포 후에는 green 태그 URL에 `/version`을 붙여 새 버전의 `BANNER=GREEN`을 확인하고, 원래 서비스 주소에서는 기존 버전이 응답하는지 비교합니다.

4-3. 새 버전으로 전환하고 롤백하기

먼저 요청 확인 스크립트를 실행합니다.

```
python scripts/watch_requests.py https://<staging 서비스 주소> --seconds 120
```

새 버전으로 전환합니다.

```
gcloud run services update-traffic friend-agent-staging \
  --region us-central1 \
  --to-tags green=100
```

문제가 발생했다고 가정하고 처음 기록한 리비전으로 되돌립니다.

```
gcloud run services update-traffic friend-agent-staging \
  --region us-central1 \
  --to-revisions <처음 리비전 이름>=100
```

여기서 핵심은 코드를 다시 수정하고 빌드하고 배포하지 않아도, 트래픽 설정만 바꿔서 되돌릴 수 있다는 것입니다.

단, 데이터베이스 구조 자체가 바뀌었다면 이전 리비전이 새 구조를 처리하지 못할 수 있습니다. 따라서 데이터베이스 변경은 롤백 가능성까지 고려해 설계해야 합니다.

4-4. 트래픽 고정 문제

트래픽을 특정 태그에 고정해 두면 새 리비전을 만들어도 새 버전이 자동으로 트래픽을 받지 않을 수 있습니다.

이때 다음 명령은 트래픽을 가장 최근 리비전으로 옮깁니다.

```
gcloud run services update-traffic friend-agent-staging \
  --region us-central1 \
  --to-latest
```

이 동작을 모르면 배포 작업은 성공했는데 사용자는 여전히 이전 버전을 보는 상황이 발생할 수 있습니다.

5. AI 개발자에게 중요한 이유

AI 서비스는 단순한 웹페이지보다 배포 후 확인할 사항이 많습니다.

예를 들어 AI 에이전트의 새 버전이 다음과 같이 바뀔 수 있습니다.

- 프롬프트 수정으로 답변 품질이 달라집니다.
- LLM API 호출 방식 변경으로 응답 속도와 비용이 증가할 수 있습니다.
- 로그인이나 대화 저장 기능에 문제가 생길 수 있습니다.
- 새로운 코드가 특정 입력에서만 오류를 일으킬 수 있습니다.

따라서 배포 성공 메시지만 보고 정상이라고 판단해서는 안 됩니다. 실제 요청이 정상적으로 처리되는지, 오류와 지연이 늘지 않는지, 핵심 기능이 유지되는지 확인해야 합니다.

롤백을 판단하는 주요 신호

확인 항목	확인할 내용
오류율	새 리비전의 5xx 오류가 증가했는가?
응답 속도	p95·p99 지연 시간이 악화되었는가?
핵심 기능	로그인, 메시지 전송, 대화 저장이 정상인가?
인스턴스 상태	시작 실패나 메모리 초과가 발생했는가?

확인 항목	확인할 내용
사용자·비용	사용자 신고나 LLM 토큰 비용이 급증했는가?

p95는 요청 100개 중 약 95개가 이 시간 이내에 응답했다는 의미입니다. 평균 응답 시간만 보면 일부 사용자에게 발생하는 심각한 지연을 놓칠 수 있습니다.

실무 원칙: 롤백 기준은 배포 전에 정합니다. 문제가 발생한 뒤 기준을 정하면 불확실한 상황에서 판단이 늦어질 수 있습니다. 또한 실패가 0건이라는 사실은 요청 수가 충분할 때에만 의미가 있습니다.

6. 사용자님의 AI 창업 프로젝트와 연결하기

사용자님이 준비하시는 1인 AI 사업과 서비스에도 이 방식은 유용합니다.

Artificial Intelligence Media Assets – Seagate Stories | Seagate US

① MSM(My-Story-Maker)

글을 분석하고 사용자의 문체를 유지하는 기능을 수정할 때 새 리비전에서 분석 결과, 문체 유지, 저장 기능을 확인한 후 전환할 수 있습니다. 기존 버전을 남겨 두면 문제가 생겼을 때 신속하게 되돌릴 수 있습니다.

AidaJet: Create, design, schedule and analyze Newsletters with AI | Product Hunt

② 뉴스레터 자동화 에이전트

뉴스 수집, 기사 선별, 근거 검증, 발행 과정 중 한 단계가 잘못 바뀌어도 모든 사용자에게 즉시 영향을 주지 않도록 새 버전을 먼저 확인할 수 있습니다. 다만 실제 발행이나 외부 전송은 테스트 환경에서 분리해야 합니다.

Vectora Digital | Consultoria digital, automatizacion, chatbots, webs y dashboards

③ AI 컨설팅 에이전트

진단 질문이나 보고서 생성 로직을 바꿀 때 테스트용 사례로 품질을 확인하고, 이전 버전과 결과를 비교한 뒤 전환할 수 있습니다. 고객 데이터와 운영 환경을 분리하는 것도 중요합니다.

1인 창업에서는 장애 대응을 전담할 운영팀이 없을 수 있습니다. 따라서 문제가 생겼을 때 빠르게 복구할 수 있는 구조 자체가 사업의 신뢰성을 높이는 경쟁력이 됩니다.

7. 핵심 요약

- 블루그린은 새 버전을 별도로 확인한 뒤 한 번에 전환하는 방식입니다.

- 카나리는 새 버전에 트래픽을 조금씩 보내며 위험을 줄이는 방식입니다.
- `-no-traffic`은 새 리비전을 만들되 일반 트래픽을 보내지 않는 핵심 옵션입니다.
- `-to-tags`, `-to-revisions`, `-to-latest`는 각각 태그, 특정 리비전, 최신 리비전으로 트래픽을 전환할 때 사용합니다.
- 트래픽 전환과 롤백은 기존 리비전을 유지한 채 트래픽 설정을 바꾸는 작업이므로 빠르게 수행할 수 있습니다.
- 카나리는 충분한 요청량과 리비전별 모니터링이 있어야 의미가 있습니다.
- 데이터베이스 변경이 있다면 이전 버전으로의 롤백 가능성도 확인해야 합니다.

8. 반드시 기억할 것

1. 배포 성공과 사용자에게 새 버전이 제공되는 것은 서로 다른 문제입니다.
2. 새 버전을 먼저 확인하고 나중에 트래픽을 전환해야 합니다.
3. 롤백할 리비전 이름을 전환 전에 기록해야 합니다.
4. 카나리 비율은 설정값과 실제 관측값이 다를 수 있습니다.
5. 실패가 없더라도 테스트 요청이 너무 적으면 안전하다고 단정할 수 없습니다.
6. 트래픽이 태그에 고정되어 있는지 확인해야 자동 배포가 의도대로 동작합니다.

9. 객관식 퀴즈 정답

정답: 2번

10분 동안 새 리비전이 받는 요청이 거의 없어 실패 0건이 안전성을 입증하지 못한다는 점이 가장 큰 문제입니다.

하루 요청이 30건인 서비스에서 새 버전으로 10%를 보내면, 단순 평균으로 하루 약 3건만 새 버전에 도달합니다. 요청 수가 적은 상황에서는 블루그린 방식으로 새 버전을 충분히 확인한 뒤 한 번에 전환하는 것이 더 적합할 수 있습니다.

10. 한 문장 요약

안전한 배포란 새 버전을 만드는 데서 끝나는 것이 아니라, 충분히 확인한 후 트래픽을 전환하고 문제가 생기면 검증된 이전 버전으로 즉시 돌아갈 수 있게 만드는 것입니다.

Chapter06

/강의정리 챕터 06. 직접 해 보기 — 블루그린 배포 자동화

1. 학습 목적

이번 강의의 핵심은 사람이 직접 실행했던 안전한 배포 절차를 GitHub Actions 워크플로로 자동화하는 것입니다.

지난 챕터에서는 Cloud Run에서 새 리비전을 트래픽 0%로 올리고, 직접 확인한 뒤 전환하거나 롤백했습니다. 이번에는 이 과정을 코드로 자동화합니다.

목표는 다음과 같습니다.

- PR을 `main`에 머지하고 production 배포를 승인합니다.
- 새 리비전이 트래픽 0%로 배포됩니다.
- 새 리비전의 상태와 커밋을 확인하는 스모크 테스트를 실행합니다.
- 테스트를 통과하면 새 리비전으로 트래픽을 100% 전환합니다.
- 문제가 발생하면 이전 리비전으로 빠르게 롤백할 수 있도록 합니다.

사람이 배포 명령을 일일이 실행하는 대신, 워크플로가 정해진 순서대로 처리하게 만드는 것입니다.

다만 자동 배포와 무중단 배포는 다릅니다. 자동으로 배포되기만 해서는 충분하지 않습니다. 새 버전이 정상인지 확인하고, 문제가 있으면 전환을 막거나 복구할 수 있어야 합니다.

2. 중학생도 이해하는 설명

배포 자동화는 자동 운전 경로를 만드는 것과 같습니다

1. PR 머지 및 승인

사람이 시작하는 단계

2. 새 리비전 배포

트래픽 0% · 새 버전 준비

3. 스모크 테스트

새 버전의 상태와 커밋 확인

4. 트래픽 100% 전환

테스트 통과 시에만 진행

테스트 실패 시

워크플로가 실패하고 전환 단계는 실행되지 않습니다. 이미 운영 중인 이전 리비전은 계속 요청을 처리합니다.

자동화의 핵심은 명령을 자동으로 실행하는 것뿐 아니라 각 단계의 성공 여부에 따라 다음 단계의 실행 여부를 통제하는 것입니다.

3. 이번 실습의 3단계

스테이지 1. 새 리비전을 트래픽 0%로 배포

지난 챕터에서 배운 명령의 핵심 옵션은 다음 두 가지입니다.

- `-no-traffic`: 새 리비전으로 일반 트래픽을 보내지 않습니다.
- `-tag green`: 새 리비전을 확인할 수 있는 태그 URL을 만듭니다.

이제 이 동작을 GitHub Actions 배포 워크플로에 반영합니다.

여기서 중요한 설계 결정이 있습니다. 기존 `deploy.yml`이 staging과 production에서 함께 사용된다면, 0% 배포를 두 환경에 모두 적용할지 production에만 적용할지 먼저 정해야 합니다. 한 환경에만 적용한다면 이후 스모크 테스트와 전환 단계에도 같은 조건을 일관되게 적용해야 합니다.

스테이지 2. 전환 전 스모크 테스트

스모크 테스트는 새 버전이 기본적인 기능을 정상적으로 수행하는지 빠르게 확인하는 검사입니다.

이번 실습에서는 두 가지를 확인합니다.

1. `/health`가 정상 응답하는가?
2. `/version`의 커밋 정보가 이번에 배포한 커밋과 일치하는가?

반드시 새 리비전의 태그 URL로 확인해야 합니다. 원래 서비스 주소는 새 리비전이 트래픽 0%인 동안 이전 리비전이 응답하기 때문입니다.

새 리비전이 막 시작되었을 때는 준비가 늦어질 수 있으므로 일정 간격으로 재시도하도록 설계할 수 있습니다. 정해진 횟수까지 실패하면 워크플로를 실패로 끝내고 전환 단계로 진행하지 않아야 합니다.

스테이지 3. 트래픽 100% 전환

스모크 테스트가 성공하면 새 리비전으로 트래픽을 모두 전환합니다.

전환 전에 현재 트래픽을 받던 리비전 이름을 로그에 남겨야 합니다. 이 정보는 롤백할 때 필요합니다.

또한 이전 챕터에서 확인했듯이 트래픽이 특정 태그나 리비전에 고정되어 있으면 다음 배포에도 영향을 줄 수 있습니다. 따라서 전환 방식과 이후 배포의 트래픽 설정이 서로 충돌하지 않는지 확인해야 합니다.

4. 주관식 퀴즈 정답

질문: 스테이지 2의 스모크 테스트를 원래 서비스 주소로 하면 어떤 일이 생길까요?

새 리비전은 트래픽 0%로 배포되었기 때문에 원래 서비스 주소로 요청하면 새 버전이 아니라 기존 리비전이 응답합니다. 따라서 기존 버전의 `/health`가 정상이고 `/version`의 커밋이 이전 값이라고 해도 새 리비전이 정상이라는 것을 증명할 수 없습니다. 새 리비전의 태그 URL로 확인해야 실제 배포한 버전의 상태와 커밋을 검증할 수 있습니다.

5. 코딩 에이전트에게 맡길 때 알아야 할 것

이번 실습에서는 코딩 에이전트가 워크플로를 작성하도록 맡겨도 됩니다. 하지만 결과물을 이해하지 못한 채 복사하는 것은 피해야 합니다.

에이전트에게 다음과 같이 요청할 수 있습니다.

`.github/workflows/deploy.yml`을 검토하고 블루그린 배포 자동화를 단계별로 구현해 줘.

1. 현재 테스트, 이미지 빌드, staging 배포, production 승인 구조와 액션 버전을 유지해 줘.
2. 새 리비전을 트래픽 0%로 배포하고 고유한 태그를 붙여 줘.
3. 태그 URL의 `/health`와 `/version`을 확인하는 스모크 테스트를 추가해 줘. `/version`의 커밋이 이번 배포 커밋과 일치해야 해.
4. 스모크 테스트가 성공한 경우에만 새 리비전으로 트래픽 100%를 전환해 줘.
5. 전환 전에 기존 트래픽을 받던 리비전 이름을 로그에 기록해 줘.
6. 기존 staging과 production 환경의 설정 및 권한 경계를 유지해 줘.
7. 실패 시 자동으로 다음 단계로 넘어가지 않도록 해 줘.

수정 전에 기존 코드와 변경안을 비교해서 보여 주고, 각 단계와 옵션의 역할을 초보자도 이해할 수 있도록 설명해 줘. 먼저 계획과 변경안을 제시하고 내가 확인하기 전에는 파일을 수정하거나 PR을 만들지 마.

추가로 다음 배포에서도 태그 이름 충돌이나 트래픽 고정 문제가 발생하지 않는지 검토해 줘. 롤백은 별도의 수동 실행 워크플로로 설계하고, 이전 리비전이 명시적으로 지정되도록

해 줘.

6. 반드시 확인해야 할 위험 요소

확인 사항	왜 중요한가요?
태그 이름 관리	이전 배포의 태그와 충돌하면 엉뚱한 리비전을 확인할 수 있습니다.
커밋 일치 여부	이번에 빌드한 코드가 실제 테스트 대상인지 확인해야 합니다.
실패 시 단계 중단	테스트가 실패했는데도 전환되면 안전장치가 의미 없어집니다.
환경 분리	staging 변경이 의도치 않게 production에 적용되면 안 됩니다.
이전 리비전 기록	롤백할 대상을 명확하게 알아야 합니다.
연속 배포	두 번째 배포에서도 0% 배포와 전환이 정상 작동해야 합니다.

특히 스모크 테스트가 성공했다고 모든 기능이 완벽하다는 뜻은 아닙니다. `/health`와 `/version`은 기본적인 확인 수단입니다. 실제 AI 서비스라면 별도로 로그인, 메시지 전송, 데이터 저장 같은 핵심 기능도 테스트해야 합니다.

7. AI 창업 프로젝트에 적용하기

사용자님이 만들고 계신 AI 에이전트에도 이 원칙을 적용할 수 있습니다.

- My-Story-Maker: 문체 분석이나 글 생성 기능을 수정한 뒤, 새 버전에서 테스트하고 문제가 없을 때 사용자에게 공개합니다.
- 뉴스레터 에이전트: 기사 수집·선별·검증 로직을 바꾼 뒤, 발행 전에 테스트합니다.
- AI 컨설팅 에이전트: 진단 보고서 생성 로직을 변경할 때 기존 결과와 새 결과를 비교하고, 정상일 때만 전환합니다.

이때 운영 데이터와 테스트 데이터를 분리하고, 외부 발행이나 고객에게 영향을 주는 작업은 테스트 단계에서 실제 실행되지 않도록 해야 합니다.

8. 오늘의 핵심 정리

이번 챕터에서 반드시 이해해야 할 것은 다음 세 가지입니다.

1. 배포와 전환은 별개입니다. 새 리비전을 올린다고 즉시 사용자가 새 버전을 사용하는 것은 아닙니다.
2. 테스트 통과 후 전환합니다. 태그 URL로 새 버전을 검증하고, 실패하면 기존 버전을 계속 운영합니다.
3. 자동화된 절차를 설명할 수 있어야 합니다. 코딩 에이전트가 작성했더라도 각 단계가 무엇을 하는지, 실패하면 어떻게 되는지 이해해야 합니다.

한 문장으로 요약하면, 안전한 배포 자동화는 새 버전을 올리는 자동화가 아니라, 검증되지 않은 버전이 사용자에게 전달되지 않도록 통제하는 자동화입니다.

Chapter07

/강의정리 챕터 07. 무중단 배포 증명과 원클릭 롤백

1. 학습 목적

이번 강의는 지난 챕터에서 만든 블루그린 배포 자동화를 완성하고, 실제로 사용자에게 장애를 일으키지 않고 배포했는지 숫자로 증명하는 것이 목표입니다.

핵심 학습은 네 가지입니다.

1. 무중단 증명: 배포 중 요청 실패가 발생했는지 확인합니다.
2. 원클릭 롤백: 문제가 생기면 GitHub Actions 버튼으로 이전 리비전으로 돌아갑니다.
3. 실패 상황 검증: 일부러 고장 난 버전을 배포해 스모크 테스트가 전환을 막는지 확인합니다.
4. 데이터베이스 호환성: 이전 버전과 새 버전이 같은 데이터베이스를 사용하는 동안 스키마 변경이 어떤 위험을 만드는지 이해합니다.

여기서 중요한 차이가 있습니다. 워크플로가 성공했다는 것은 작업이 정상 종료되었다는 뜻이지, 사용자에게 장애가 없었다는 뜻은 아닙니다. 배포 과정의 요청 결과까지 관측해야 무중단 여부를 확인할 수 있습니다.

2. 중학생도 이해하는 설명

2-1. 무중단 배포는 어떻게 증명할까요?

병원 접수 시스템을 새 버전으로 바꾼다고 생각해 보겠습니다. 기존 접수 시스템을 유지한 채 새 시스템을 준비하고, 새 시스템이 정상인지 확인한 다음 접수 업무를 넘깁니다.

이때 확인할 질문은 단순히 “새 시스템이 설치되었나요?”가 아닙니다.

- 전환하는 동안 접수 요청이 실패하지 않았나요?
- 전환 전에는 기존 시스템이, 전환 후에는 새 시스템이 응답했나요?
- 새 시스템이 실제로 정상 동작했나요?

이번 실습에서는 `scripts/watch_requests.py`로 이 과정을 관측합니다.

```
python scripts/watch_requests.py https://<production 서비스 주소>
```

배포 승인 직전에 실행하고 전환이 끝날 때까지 요청 결과를 확인합니다. 스크립트는 최대 5분 동안 실행되므로, 배포가 그보다 오래 걸리면 다시 실행해야 합니다.

전체 요청 수

배포 중 실제로 얼마나 많은 요청을 확인했는가?

리비전별 응답 수

기존 리비전과 새 리비전이 모두 요청을 처리했는가?

실패 수

관측한 요청 중 실패가 몇 건 발생했는가?

실습에서 이전 리비전과 새 리비전의 응답이 모두 관측되고 실패가 0건이라면, 관측된 요청에 대해서는 끊김 없이 전환되었다는 근거가 됩니다.

다만 이것만으로 모든 사용자의 무중단을 완벽히 증명하는 것은 아닙니다. 요청 수가 충분한지, 실제 핵심 기능도 정상인지, 실패가 네트워크 문제인지 배포 문제인지 함께 살펴봐야 합니다.

3. 스테이지 5 — 원클릭 롤백

원클릭 롤백이란?

새 버전에 문제가 생겼을 때 GitHub 저장소의 Actions 탭에서 `Run workflow`를 눌러, 이전 리비전으로 트래픽을 되돌리는 방식입니다.

지난 챕터에서는 명령어를 직접 실행했습니다. 이번에는 그 명령을 별도의 `.github/workflows/rollback.yml` 파일에 넣어 필요할 때 실행할 수 있도록 만듭니다.

① 새 버전에서 문제 발생

오류 증가, 핵심 기능 실패 등

② GitHub Actions에서 실행

Run workflow 버튼으로 롤백 작업을 시작합니다.

③ 이전 리비전으로 전환

트래픽 100%를 지정한 리비전으로 옮기고 `/version`으로 확인합니다.

롤백 워크플로를 설계할 때의 선택

- 되돌아갈 리비전 지정: 버튼을 누를 때 리비전 이름을 직접 입력하면 의도한 대상을 명확하게 선택할 수 있습니다.
- 이전 리비전 자동 탐색: 편리하지만, 트래픽 설정과 배포 기록을 잘못 해석하면 엉뚱한 버전으로 돌아갈 위험이 있습니다.
- production 승인: 승인 절차를 유지하면 무단 실행 위험을 줄일 수 있지만, 장애 대응이 늦어질 수 있습니다. 승인 여부는 조직의 권한 관리와 긴급 대응 정책을 고려해 결정해야 합니다.

개인 프로젝트라도 롤백은 강력한 운영 권한입니다. GitHub에서 누가 실행할 수 있는지 제한하고, 배포에 사용 중인 인증 방식과 필요한 권한을 재사용하는 것이 중요합니다.

4. 스테이지 6 — 일부러 고장 난 버전으로 시험하기

안전장치는 정상 상황에서만 확인해서는 부족합니다. 실제로 장애를 막아내는지 시험해야 합니다.

이번에는 `/health`가 항상 실패하도록 만든 버전을 배포합니다.

기대하는 결과는 다음과 같습니다.

1. 워크플로가 실패합니다.
2. 실패 지점은 배포 단계 또는 스모크 테스트 단계입니다.
3. 트래픽 전환 단계는 실행되지 않습니다.
4. 기존 리비전이 계속 트래픽 100%를 받습니다.
5. 요청 확인 스크립트에서 실패가 0건으로 관측됩니다.
6. GitHub의 Revert 기능으로 잘못된 변경을 되돌린 뒤 정상 배포가 다시 성공합니다.

특히 Cloud Run의 시작 상태 확인이 `/health`를 검사하도록 설정되어 있다면, 새 리비전이 준비되지 않아 스모크 테스트 전에 배포 단계에서 실패할 수도 있습니다. 어느 단계에서 실패하더라도 새 버전으로 트래픽이 전환되지 않았다면 안전장치가 제 역할을 한 것입니다.

단, 실제 production에서 고장 난 버전을 시험하기보다는 격리된 staging 환경에서 먼저 검증하는 것이 안전합니다.

5. 객관식 퀴즈 정답

정답: 2번

`/health`가 항상 실패하는 버전을 머지했는데도 워크플로가 성공하고 `/version`이 새 커밋으로 바뀌었다면, 우선 스모크 테스트가 제대로 작동했는지 확인해야 합니다.

가능성이 높은 원인은 두 가지입니다.

- 스모크 테스트가 새 리비전의 태그 URL이 아니라 기존 리비전이 응답하는 원래 서비스 주소를 확인했습니다.
- 테스트 실패가 워크플로의 실패로 처리되지 않아 다음 전환 단계까지 실행되었습니다.

즉, 테스트를 실행했다는 사실만으로는 충분하지 않습니다. 올바른 대상을 검사하고, 실패하면 전환이 중단되는 구조인지 확인해야 합니다.

6. 데이터베이스 스키마 변경이 위험한 이유

이번 강의에서 가장 중요한 심화 개념 중 하나입니다.

Cloud Run의 이전 리비전과 새 리비전은 배포 전환 중에 같은 Supabase 데이터베이스를 사용할 수 있습니다. 이때 테이블의 열 이름을 바꾸면 문제가 발생할 수 있습니다.

예를 들어 기존 `reminders` 테이블에 `reminder_text`라는 열이 있다고 가정하겠습니다.

기존 리비전

`reminder_text` 열을 사용

이전 코드가 기존 열 이름으로 데이터를 읽고 씁니다.

새 리비전

`message` 열을 사용하도록 변경

새 코드는 변경된 열 이름을 기대합니다.

문제 발생 가능

두 리비전이 함께 실행되는 동안 데이터베이스가 한쪽 코드와 호환되지 않으면 오류가 발생할 수 있습니다. 롤백해도 데이터베이스 구조가 자동으로 원래대로 돌아가지는 않습니다.

안전하게 스키마를 변경하는 방법

일반적으로는 확장 → 코드 전환 → 정리 순서가 안전합니다.

1. 확장: 새 열을 추가하되 기존 열은 유지합니다.
2. 호환 코드 배포: 이전 코드와 새 코드가 모두 동작할 수 있게 합니다. 필요하면 양쪽 열을 함께 기록하고 데이터를 동기화합니다.
3. 코드 전환: 새 코드가 새 열을 사용하도록 전환합니다.
4. 정리: 이전 리비전으로 돌아갈 필요가 없고 데이터가 검증된 뒤에만 이전 열을 제거합니다.

이러한 방식은 블루그린뿐 아니라 카나리 배포에서도 중요합니다. 카나리로 새 리비전에 트래픽을 10%만 보내더라도 두 버전이 같은 테이블을 사용하기 때문입니다.

7. 선택 과제 — 기본과 도전

기본: Discord 또는 Slack 알림

배포 성공, 배포 실패, 롤백 완료 시 알림을 보냅니다. 웹훅 URL은 GitHub Secrets에 저장하고, 워크플로에는 시크릿 이름만 사용합니다.

완료 기준: 고장 난 버전이 배포 단계에서 중단되었을 때 실패 알림이 오고, 워크플로 파일에 웹훅 URL이 노출되지 않습니다.

도전: 카나리 자동 승격

새 리비전에 10%의 트래픽을 보내고 일정 시간 관측한 뒤, 오류율과 지연 시간 기준을 통과하면 50%, 100%로 확대합니다. 기준을 넘으면 이전 리비전으로 롤백합니다.

완료 기준: 정상 버전은 단계적으로 확대되고, 문제가 있는 버전은 정해진 기준에 따라 전환이 중단되거나 롤백됩니다.

카나리 자동 승격은 블루그린보다 구현 난도가 높습니다. 특히 요청 수가 적으면 오류율과 지연 시간 통계가 불안정할 수 있으므로, 충분한 관측량과 명확한 판정 기준을 먼저 마련해야 합니다.

8. 사용자님의 AI 창업 준비와 연결하기

이번 학습은 단순히 Cloud Run을 다루는 기술을 넘어서, AI 서비스를 실제 고객에게 제공할 때 필요한 운영 체계를 만드는 과정입니다.

사용자님의 프로젝트	적용할 수 있는 운영 원칙
My-Story-Maker	글 생성 기능을 수정한 뒤 문체 분석·생성·저장 기능을 확인하고 전환합니다.
뉴스레터 에이전트	수집·선별·검증 단계의 변경을 시험하고, 발행 단계의 오류를 감지해 중단합니다.
AI 컨설팅 에이전트	보고서 생성 품질을 비교하고, 문제가 있으면 이전 버전으로 복구합니다.

1인 AI 사업에서는 장애를 처리할 전담 운영팀이 없을 수 있습니다. 그래서 처음부터 다음 세 가지를 마련하는 것이 중요합니다.

- 감지: 오류가 발생했다는 사실을 알 수 있어야 합니다.
- 복구: 문제가 생기면 이전 버전으로 돌아갈 수 있어야 합니다.
- 증명: 실제 요청 결과와 로그를 통해 정상 작동 여부를 확인할 수 있어야 합니다.

이 세 가지가 갖춰지면 혼자 운영하더라도 서비스를 더 체계적으로 관리할 수 있습니다.

9. 반드시 기억할 것

- 배포 성공과 무중단 배포는 다릅니다. 배포 중 실제 요청 결과를 확인해야 합니다.
- 요청 확인 스크립트의 실패 시각을 GitHub Actions 로그와 비교해 원인을 분석합니다.
- 원클릭 롤백은 이전 리비전의 이름과 실행 권한을 명확히 관리해야 합니다.
- 고장 난 버전으로 시험해 스모크 테스트가 실제로 전환을 막는지 검증합니다.
- 데이터베이스 스키마는 Cloud Run 리비전처럼 단순히 트래픽을 되돌린다고 자동 복구되지 않습니다.
- 카나리 자동 승격은 충분한 관측 데이터와 사전에 정의된 롤백 기준이 있어야 합니다.
- 시크릿을 워크플로 파일에 직접 기록하지 않고, 최소 권한 원칙을 유지합니다.

10. 한 문장 요약

안전한 AI 서비스 운영은 새 버전을 배포하는 기술만이 아니라, 장애를 감지하고, 전환을 통제하고, 빠르게 복구하며, 그 결과를 숫자로 증명하는 체계를 만드는 것입니다.

Chapter08

챕터 08 강의 정리 — Cloud Run 무중단 배포와 운영 자동화

1. 학습 목적

이번 강의의 목적은 단순히 AI 서비스를 배포하는 것을 넘어, 서비스가 운영되는 중에도 안전하게 새 버전을 적용하고, 문제가 생기면 빠르게 복구할 수 있는 구조를 이해하는 것입니다.

Cloud Run의 기본 기능과 직접 설정해야 하는 항목을 구분하고, 블루그린·카나리 배포와 워크플로 자동화를 통해 무중단 배포를 구현하는 방법을 배웠습니다.

2. 중학생도 이해할 수 있는 핵심 개념

① Cloud Run이 알아서 하는 일과 내가 정하는 일

Cloud Run은 교통정리를 담당하는 교통경찰과 비슷합니다. 요청을 여러 리비전으로 보내고, 이전 인스턴스를 정리합니다.

하지만 새 버전이 실제로 요청을 받을 준비가 되었는지는 별도로 판단해야 합니다.

- Cloud Run이 담당: 로드밸런싱, 트래픽 분배, 이전 인스턴스 종료 처리

- 내가 설정: 새 버전의 준비 상태를 확인하는 스타트업 프로브와 확인 경로

기본 포트 확인 대신 `/health`를 준비 상태 확인에 사용하자, 시작이 늦은 버전에서도 준비가 완료될 때까지 기다릴 수 있어 요청 실패가 0건으로 나타났습니다.

② 대화가 사라지지 않는 서비스

새 버전이 배포될 때마다 서버 안에만 대화 내용을 저장한다면, 이전 서버가 종료될 때 대화가 사라질 수 있습니다.

이를 해결하기 위해 staging과 production을 Supabase 데이터베이스에 연결했습니다.

- 대화 내용을 서버 외부의 데이터베이스에 저장합니다.
- 새 리비전과 이전 리비전이 같은 데이터를 볼 수 있습니다.
- 데이터베이스 키는 코드에 직접 넣지 않고 시크릿으로 관리합니다.

다만, 두 버전이 같은 테이블을 사용하는 만큼 데이터 구조를 바꿀 때는 이전 버전과의 호환성도 고려해야 합니다.

③ 블루그린과 카나리 배포

Mastering Blue-Green Deployment. Introduction | by Deepal Jain | Medium

블루그린 배포

새 버전을 트래픽 0%로 배포하고 태그 URL로 미리 확인한 다음, 문제가 없으면 트래픽을 전환합니다. 문제가 생기면 이전 버전으로 되돌립니다.

적합한 상황: 새 버전을 충분히 확인한 뒤 한 번에 전환하고 싶을 때

MACHINE LEARNING MODEL DEPLOYMENT PATTERNS | by Gideon oboro | Medium

카나리 배포

새 버전에 먼저 10%처럼 적은 트래픽을 보내고, 결과가 괜찮으면 50%, 100%로 확대합니다.

적합한 상황: 실제 요청으로 새 버전의 안정성을 단계적으로 확인하고 싶을 때

이번 강의에서는 특정 리비전에 트래픽이 고정되면 다음 배포가 자동으로 트래픽을 받지 못할 수 있다는 점도 확인했습니다. 따라서 배포할 때마다 트래픽 설정을 함께 점검해야 합니다.

④ 손으로 하던 배포를 워크플로로 자동화하기

기존에는 사람이 직접 수행하던 작업을 워크플로의 단계로 옮겼습니다.

1. 새 리비전을 트래픽 0%로 배포합니다.
2. 태그 URL로 새 리비전의 `/health`, `/version` 등을 확인합니다.
3. 테스트가 성공하면 트래픽을 전환합니다.
4. 실패하면 전환하지 않고 멈춥니다.
5. 문제가 생기면 이전 리비전으로 롤백합니다.

마지막으로 요청 확인 스크립트를 실행해 실제 요청의 실패가 0건인지 확인했습니다. 무중단 배포는 단순히 오류가 없을 것이라고 기대하는 것이 아니라, 실제 요청 결과로 확인하는 것이 핵심입니다.

3. 비유로 이해하기

새로운 식당 메뉴를 출시한다고 생각해 보겠습니다.

- 스타트업 프로브: 새 주방이 손님을 받을 준비가 되었는지 확인하는 점검
- 블루그린: 새 메뉴를 미리 준비하고 확인한 뒤 전체 메뉴를 교체하는 것
- 카나리: 일부 손님에게만 새 메뉴를 제공해 반응을 확인하는 것
- Supabase: 주방이 바뀌어도 주문 내역이 남아 있는 공용 주문 시스템
- 워크플로: 준비·검사·출시·복구 과정을 체크리스트에 따라 자동으로 수행하는 시스템

새 주방이 준비되지 않았는데 손님을 보내면 문제가 생깁니다. 반대로 준비 여부를 확인하고, 일부 손님에게 먼저 제공하며, 문제가 있으면 이전 메뉴로 돌아갈 수 있다면 훨씬 안전합니다.

4. 실제 활용 예시

사용자님의 뉴스레터 에이전트나 MSM(My-Story-Maker)에 적용할 수 있습니다.

- 뉴스레터 에이전트: 새로운 기사 수집·평가 로직을 0% 배포 후 확인하고, 이상이 없을 때 운영 트래픽을 전환합니다.
- MSM: 새로운 글 분석 또는 웹소설 생성 기능을 카나리 방식으로 일부 사용자에게 먼저 제공할 수 있습니다.
- AI 컨설팅 서비스: 새 버전에서 상담 기록이 사라지지 않도록 데이터베이스에 저장하고, 테스트 실패 시 이전 버전으로 복구합니다.

서비스가 복잡해질수록 배포 자동화와 데이터 보호는 기능 개발만큼 중요한 운영 역량이 됩니다.

5. AI 개발자·1인 창업가가 알아야 하는 이유

코딩을 AI 에이전트에게 맡기더라도, 운영 구조를 이해하지 못하면 잘못된 설정을 발견하기 어렵습니다.

특히 다음 세 가지는 반드시 이해해야 합니다.

- 준비 상태와 실행 상태는 다릅니다. 서버가 실행 중이어도 요청을 처리할 준비가 안 되었을 수 있습니다.
- 코드 롤백과 데이터 롤백은 다릅니다. 이전 리비전으로 돌아가도 이미 변경된 데이터베이스 구조까지 자동으로 복구되지는 않습니다.
- 자동화와 검증은 다릅니다. 배포가 자동으로 완료되었다는 사실만으로 무중단이 증명되지는 않습니다.

6. 주관식 퀴즈 답안 — 회고

생각이 달라진 부분

처음에는 한 번에 전환하는 것보다 트래픽을 10%만 보내는 카나리 배포가 항상 더 안전하다고 생각했습니다. 하지만 오늘 실습하면서 배포 방식뿐 아니라 새 버전의 준비 상태를 정확하게 확인하고, 문제가 발생했을 때 전환을 멈추거나 롤백할 수 있는 구조가 중요하다는 점을 알게 되었습니다.

특히 준비가 늦은 버전에서도 스타트업 프로브를 적절하게 설정하자 요청 실패가 0건으로 나타났고, 태그 URL을 이용하면 새 리비전에 운영 트래픽을 보내기 전에 미리 확인할 수 있었습니다. 이를 통해 배포 전에 충분히 검증하는 것만으로도 위험을 줄일 수 있다는 점을 배웠습니다.

또한 카나리 배포는 위험을 줄이는 데 도움이 되지만, 트래픽이 적으면 10%에 해당하는 요청 자체가 너무 적어 문제를 발견하기 어려울 수 있습니다. 따라서 카나리가 항상 정답인 것은 아니며, 서비스의 요청량과 위험 수준에 따라 블루그린 배포와 카나리 배포를 선택해야 한다고 생각합니다.

그대로인 부분

여전히 새 버전을 한 번에 전환하는 것보다 단계적으로 전환하는 방식이 더 안전하다는 생각은 유지하고 있습니다. 다만 그 안전성은 트래픽 비율만으로 결정되는 것이 아니라, 새 리비전의 사전 검증, 오류 관측, 롤백 준비 여부에 따라 달라진다는 점을 더 분명하게 이해하게 되었습니다.

또한 트래픽을 특정 리비전에 고정하면 다음 배포가 트래픽을 받지 못할 수 있다는 사실을 직접 확인했습니다. 자동화된 워크플로도 스모크 테스트가 실패하면 전환하지 않도록 설계

해야 하며, 실제 요청 확인 스크립트로 실패가 0건인지 검증해야 한다는 점도 중요하게 느꼈습니다.

최종 결론

안전한 배포의 핵심은 무조건 10%씩 트래픽을 보내는 것이 아니라, 새 버전이 준비되었는지 확인하고, 적절한 방식으로 전환하며, 문제가 생기면 신속하게 복구할 수 있도록 만드는 것입니다. 앞으로 제가 만드는 AI 서비스에도 서비스 규모와 위험 수준에 맞는 배포 전략과 자동 검증·롤백 체계를 적용하겠습니다.

7. 핵심 요약

- Cloud Run의 기본 기능과 직접 설정해야 할 준비 상태 검사를 구분합니다.
- Supabase 같은 외부 데이터베이스에 상태를 저장하면 리비전이 바뀌어도 데이터를 유지할 수 있습니다.
- 블루그린은 사전 검증 후 전환하는 방식이고, 카나리는 실제 트래픽을 단계적으로 늘리는 방식입니다.
- 트래픽 고정 설정은 다음 배포를 방해할 수 있으므로 주의해야 합니다.
- 워크플로는 배포뿐 아니라 검증, 전환 중단, 롤백까지 포함해야 합니다.
- 무중단은 배포 완료 메시지가 아니라 실제 요청 결과로 검증합니다.

8. 반드시 기억할 것

안전한 배포 = 준비 상태 확인 + 사전 검증 + 적절한 트래픽 전환 + 빠른 롤백 + 실제 요청 검증

한 문장 요약: 무중단 배포란 새 버전을 올리는 기술이 아니라, 문제가 생겨도 사용자의 서비스 이용을 지킬 수 있도록 검증과 복구까지 설계하는 운영 기술입니다.